A Summary of Digital Image Processing Applications and Suitable Image Features

Malvino Rosana Pasquale
Student Number: 25/569865/PA/24015
Digital Image Processing Course
Assignment 4

***Abstract*—Digital image processing offers practical solutions for extracting information, improving image quality, and supporting visual inspection. This report summarizes five application cases: object counting, scanned-document enhancement, fruit ripeness assessment, surface-defect inspection, and impulse-noise removal. Each case is discussed in terms of its objective, appropriate processing techniques, suitable image features, and practical limitations. Intensity and shape are useful for separating and counting objects; local contrast supports document binarization; color describes visible ripeness changes; texture and edges indicate surface irregularities; and neighborhood statistics help identify impulse noise. The discussion emphasizes that features should be selected according to the physical characteristics of the problem. This is a conceptual review rather than an implementation or experimental evaluation, and no numerical performance claims are made.**

***Index Terms*—digital image processing, feature selection, segmentation, color, texture, shape**

I. Introduction

Digital image processing uses mathematical operations to improve an image or extract useful information from it. An image can be treated as a collection of pixel values, but solving a practical problem requires understanding what those values represent. A dark region could be printed text, a shadow, a damaged surface, or simply an object with a dark color. The same processing operation is therefore not equally suitable for every application.

An image feature is a measurable property that describes a pixel, a neighborhood, or an entire object. Common examples include intensity, color, texture, area, and boundary shape. Features provide a connection between the visual appearance of an image and the information required by a task. Standard image-processing methods include filtering, thresholding, edge detection, morphology, and connected-component analysis [1].

The purpose of this report is to summarize several cases that can be addressed with these techniques and identify suitable features for each case. The discussion focuses on accessible methods appropriate for an introductory course. Proposed workflows explain how a problem could be approached; they do not represent tested systems. The two sample images in Fig. 1 are included only to illustrate typical input conditions, not to demonstrate experimental results.

II. Feature Categories and Selection Principles

A. Intensity and Color

Intensity measures brightness and is useful when foreground objects differ from their background. A histogram summarizes the distribution of intensities and can help determine whether threshold-based separation is reasonable. Otsu's method selects a threshold by maximizing between-class variance [2]. However, a single threshold can fail when object and background intensities overlap.

Color features describe information that grayscale conversion would remove. Hue and saturation can be useful when the visual difference between categories is mainly chromatic. A color histogram records how frequently different colors occur, while channel means provide a simpler summary. These features are affected by illumination and camera white balance, so consistent acquisition is important.

B. Shape, Texture, and Local Statistics

Shape features describe the geometry of a segmented region. Area measures region size, perimeter describes boundary length, and aspect ratio distinguishes elongated objects from compact ones. Circularity is commonly expressed as

$$C = \frac{4\pi A}{P^2}, \tag{1}$$

where A is area and P is perimeter. A continuous circle has circularity equal to one, although pixel sampling affects measured values.

Texture features describe spatial patterns rather than individual pixel values. Local variance indicates intensity variation, while gray-level co-occurrence matrix (GLCM) features characterize relationships between neighboring gray levels. Haralick et al. established a widely used framework for these texture descriptors [3]. Local statistics such as the mean and median are also useful for distinguishing isolated outliers from consistent neighborhoods.

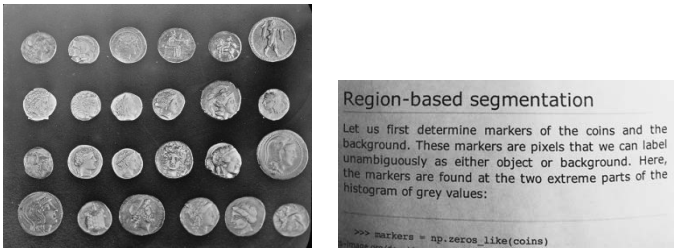
III. Case I: Counting Separated Objects

A. Problem Summary and Suitable Techniques

Object counting is useful in inventory checking, packaging, and classroom examples involving coins or seeds. The objective is to identify each visible object and count it once. Under a simple imaging setup, the objects are separated and placed on a background with a different brightness.

TABLE I
SUMMARY OF APPLICATION CASES, SUITABLE FEATURES, AND PRACTICAL LIMITATIONS

Case	Candidate techniques	Suitable features	Main limitation
Object counting	Thresholding, morphology, component labeling	Intensity, area, circularity, centroid	Touching objects, shadows, and scale variation
Document enhancement	Background correction, adaptive binarization	Local intensity, contrast, stroke width	Thin strokes and textured backgrounds
Fruit ripeness	Color-space conversion, color segmentation	Hue, saturation, color-region ratio, spots	Lighting variation and species dependence
Surface inspection	Edge detection, reference comparison, texture analysis	Gradient, elongation, variance, GLCM features	Normal patterns may resemble defects
Impulse denoising	Median or switching median filtering	Extreme values, local median deviation	Detail loss and dense corruption



(a) Separated coins (b) Scanned text

Fig. 1. Illustrative source images from the scikit-image collection [4]. The coins illustrate object/background separation; the page illustrates uneven illumination. These are input examples, not processed results.

A suitable workflow is grayscale conversion, light smoothing, thresholding, morphological cleanup, and connected-component labeling. Thresholding produces a foreground mask. Morphological operations can remove small fragments and fill small holes. Each remaining connected region is then treated as a candidate object. If objects touch, distance-based watershed segmentation can be considered, but it introduces additional assumptions and parameters.

B. Suitable Features and Their Roles

Intensity provides the initial separation between object and background. **Area** helps reject tiny regions that are unlikely to be complete objects. **Circularity** is useful when the objects are approximately round, as with coins. The **centroid** records the position of each detected object and helps visualize whether a region corresponds to one physical item.

These features are suitable because counting requires distinct spatial regions, not necessarily object identity. However, area in pixels is not physical size unless the image scale is calibrated. Overlapping objects, shadows, and bright background patches can cause merging or missing detections. A system should therefore be evaluated against manually counted objects across multiple images rather than assuming that every connected region is a valid object.

IV. CASE II: ENHANCING SCANNED DOCUMENTS

A. Problem Summary and Suitable Techniques

Scanned pages and photographs of documents may contain shadows, uneven brightness, and weak contrast. The objective is to separate printed text from the page background so that the document becomes easier to read or prepare for optical character recognition (OCR).

A suitable approach includes grayscale conversion, optional background correction, and binarization. Global thresholding can work with evenly illuminated paper. Adaptive thresholding is more appropriate when background brightness varies because it determines a threshold from each local neighborhood [5]. Small morphological operations may remove isolated speckles, but aggressive cleanup can erase punctuation or thin strokes.

B. Suitable Features and Their Roles

Local intensity describes the brightness around a pixel, while **local contrast** distinguishes dark strokes from nearby paper. **Stroke width** and **component area** can help distinguish likely text structures from small isolated noise. **Edge orientation** or alignment of text lines can also support document deskewing.

Local features are suitable because an illuminated part of a page and a shaded part may require different thresholds. However, connected components do not necessarily correspond to characters: a dot can form a separate component, and neighboring letters can merge. A clean-looking binary page does not establish recognition accuracy. Appropriate evaluation would compare the result with a reference mask or measure character and word errors against a verified transcription.

V. CASE III: ASSESSING VISIBLE FRUIT RIPENESS

A. Problem Summary and Suitable Techniques

Fruit sorting is a practical agricultural application. For fruits whose skin color changes with ripeness, such as bananas or tomatoes, image processing can support an initial visual assessment. The objective is to group fruit by visible appearance, not to directly measure taste, sugar content, or internal quality.

A basic workflow is to isolate the fruit from the background, convert the image into a suitable color representation, and summarize the colors inside the fruit mask. Hue–saturation–value (HSV) can make chromatic information easier to interpret than raw RGB values. Simple rules may then describe the proportion of green, yellow, or red regions, depending on the fruit species and the intended categories.

B. Suitable Features and Their Roles

Hue distribution captures dominant skin color, while **saturation** helps separate vivid skin colors from pale highlights. The **proportion of color regions** is more informative than a single mean when a fruit has mixed ripeness. **Dark-spot area** and **texture variation** may provide additional information about visible blemishes or advanced ripening.

These features match the visible changes being examined, but their interpretation must be species-specific. A green apple is not necessarily unripe, and a shadow is not necessarily a bruise. Glossy reflections, background leakage, and inconsistent lighting can distort color measurements. Controlled illumination and color calibration are therefore important. Any claim about internal ripeness would require independent measurements rather than image appearance alone.

VI. Case IV: Inspecting Surface Defects

A. Problem Summary and Suitable Techniques

Manufacturing inspection often aims to locate scratches, cracks, stains, or other irregularities on a surface. Digital image processing can help flag suspicious regions for review, particularly when the normal surface has a consistent appearance.

A suitable workflow includes controlled image acquisition, brightness normalization, edge detection, and texture analysis. An edge detector can emphasize abrupt changes, while comparison with a normal reference image can reveal unexpected structures. Morphological operations can connect nearby crack fragments or remove isolated detections. The result is a candidate defect map rather than an automatic guarantee of product quality.

B. Suitable Features and Their Roles

Gradient magnitude highlights sharp transitions that may correspond to scratches or cracks. **Elongation**, **length**, and **orientation** help describe narrow linear defects. **Local variance** and **GLCM contrast** describe texture changes; **GLCM homogeneity** describes how strongly neighboring gray levels concentrate near similar values [3].

These features are appropriate because defects often interrupt the expected geometry or texture of the material. However, normal grain, seams, printed patterns, and reflections may also produce strong edges. The normal surface must therefore be characterized before selecting thresholds. Inspection performance should consider both missed defects and false alarms, using independently labeled examples of acceptable and defective surfaces.

VII. Case V: Removing Impulse Noise

A. Problem Summary and Suitable Techniques

Impulse noise appears as isolated extreme pixels, often described as salt-and-pepper noise. It can reduce image readability and interfere with later segmentation. The objective is to suppress abnormal pixels while preserving real edges and fine structures.

Median filtering is a suitable introductory method because it replaces a pixel with the median of its neighborhood. Unlike averaging, the median is less influenced by a small number of extreme values. An adaptive or switching approach can instead replace only pixels considered suspicious. Filter size should be chosen carefully because a large neighborhood can remove important details.

B. Suitable Features and Their Roles

Extreme intensity provides a clue that a pixel may be corrupted. **Deviation from the local median** indicates whether that pixel disagrees strongly with its surroundings. **Neighborhood consistency** helps distinguish an isolated outlier from a legitimate bright or dark region. These are restoration cues rather than object-classification features.

An extreme pixel is not automatically noise: a real highlight or a small dark feature can have the same value. Dense corruption can also overwhelm a neighborhood median. Median filtering is especially appropriate for sparse impulses, but it is not universally optimal for all noise distributions. When a clean reference exists, restoration may be evaluated using mean squared error or peak signal-to-noise ratio; these are evaluation metrics, not features available during ordinary processing.

VIII. Discussion

Table I shows that suitable features depend on the objective. Shape is important when identifying individual objects, but it is less relevant when deciding whether one pixel is corrupted. Color can distinguish visible ripeness changes but contributes little to a grayscale scanned page. Texture can reveal a damaged surface, yet a strongly textured background can make document enhancement more difficult.

Feature selection also depends on acquisition conditions. Region area changes with camera distance, color changes with illumination, and texture descriptors depend on image resolution and neighborhood size. Reliable acquisition and normalization may therefore improve consistency more than adding many unrelated features. Selecting a small set of meaningful features also makes a student-level analysis easier to explain.

Finally, processing techniques, image features, and evaluation metrics should be kept distinct. Thresholding is an operation; intensity is an input property; circularity is a region descriptor; and count error is an evaluation measure. Similarly, a median is a local statistic, while reference-based restoration scores assess the resulting image. A future implementation would require representative images and appropriate reference labels, but this conceptual summary does not claim experimental validation.

IX. Conclusion

Digital image processing can support a wide range of practical tasks using understandable techniques. Object counting benefits from intensity and geometry, document enhancement from local brightness and contrast, fruit assessment from color, surface inspection from texture and edges, and impulse-noise removal from neighborhood statistics. No single feature is

best for all applications. Suitable choices follow the visible characteristics of the target problem and must account for illumination, scale, background, and noise. The five cases demonstrate how to justify feature selection without requiring coding or unsupported numerical results.

REFERENCES

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[3] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973.

[4] scikit-image contributors, "Sample images: skimage.data, version 0.24," https://scikit-image.org/docs/0.24.x/api/skimage.data.html, accessed: Nov. 9, 2026.

[5] ——, "Thresholding: skimage.filters, version 0.24," https://scikit-image.org/docs/0.24.x/api/skimage.filters.html, accessed: Nov. 9, 2026.